

スキルシート

- このスキルシートのリンク - <https://github.com/sugurutakahashi-1234/skills-sheet>

基本情報

- 所在: 東京
- 現在のポジション: [株式会社ZENSHIN](#) CTO / FDE
 - CTO: 自社の AI マッチングシステムの開発・運用と、社内 IT 全般
 - FDE: 顧客の業務に入り込み、AI ツールを自ら実装して導入まで担当
- エンジニア歴: 10年目 (2017年～)
- 外部リンク:
 - GitHub - <https://github.com/sugurutakahashi-1234>
 - X - https://x.com/suguru_takaha4
 - Qiita - <https://qiita.com/sugurutakahashi12345>
 - Zenn - <https://zenn.dev/ikuraikura>

強み

- FDE (AI 活用の相談・提案～実装・導入)**
 - 経営層から実装担当者まで、相手に合わせた AI 活用の相談・提案を担当
 - 案件獲得の営業・商談から、顧客業務に入り込んだ AI ツールの実装・導入までを一気通貫で担当
- AI システム開発 (RAG / MCP / AI エージェント)**
 - 人材マッチングや開発案件の自動評価のシステムを、設計から実運用まで単独でフルスタック開発
 - 非エンジニアの営業が Claude Code / Codex で実業務を回せる状態まで導入
 - AI 判定と人間判断の乖離を本番データで分析し、プロンプトを継続改善する運用ループを確立
- 0 → 1 リードエンジニア経験 (5件)**
 - アーキテクチャ選定・CI/CD 構築・チーム運用の設計など、プロジェクト立ち上げの技術リードを担当 (現職 CTO でのプロダクト立ち上げを含む)
- KPI 起点のグロース施策**
 - ビジネス KPI (ARPU・LTV・CTR) を基点に、施策の企画・A/B テスト検証・実装までを一気通貫で推進
- フルスタック実装力**
 - iOS・Android・Web のフロントエンド (Swift / React Native / Flutter / Next.js / Astro) からサーバーサイドまで単独で実装可能
 - マルチクラウドの IaC (Terraform × GCP / Cloudflare / AWS) まで一人で構築・運用
- OSS 開発・技術発信**
 - 自作 OSS (TypeScript 製 CLI ツール) を npm / Homebrew / GitHub Actions Marketplace で配布
 - 独自 iOS アーキテクチャ「FIA」の考案・公開や、Zenn での技術記事発信を継続

技術スタック

AI

- AI システム開発**
 - RAG / Embedding / ベクトル検索 (Cloudflare Vectorize, Workers AI)
 - MCP サーバー開発 (@modelcontextprotocol/sdk v2, Better Auth OAuth), WebMCP
 - Vision LLM, Cloudflare Workflows / Queues
 - ユースケース別 LLM モデル選定 (Kimi K2 / gpt-oss / GLM / Llama, 自作評価スクリプトで比較検証)
 - プロンプト運用 (バージョン管理・本番データでの継続改善)
- 生成 AI 活用**
 - Claude Code (Skills / MCP / エージェント運用 / GitHub Actions 自動レビュー), Codex CLI, Claude Code Managed Agents

言語 / バックエンド

- 言語
 - TypeScript, Swift, Dart, Python, PHP, Java
- サーバーサイド
 - API 設計: REST API 設計, GraphQL 設計
 - 実装: Node.js, Hono, oRPC, PHP, Java
- データベース
 - 設計: RDB のテーブル設計
 - ORM: Prisma, Drizzle
 - RDBMS: PostgreSQL, MySQL, SQLite (Cloudflare D1)
 - NoSQL / ベクトル DB: Firestore, Cloudflare Vectorize
- 認証 / 認可
 - プロトコル / 仕様: OAuth 2.0, JWT
 - 実装 / サービス: Better Auth, LINE Login, Cognito, Firebase Authentication, Cloudflare Access
 - クラウド間連携: Workload Identity Federation (OIDC)

クラウド / インフラ

- クラウド
 - **Cloudflare**
 - コンピュート / 配信: Workers, Pages, Browser Rendering
 - AI: Workers AI, AI Gateway, Vectorize
 - データ / 非同期処理: D1, R2, KV, Queues, Workflows
 - ネットワーク / セキュリティ: DNS, Access, Turnstile, Email Routing, Email Sending
 - **Google Cloud**
 - Cloud Run, GCS, IAM, Cloud Identity, Workload Identity Federation
 - **AWS**
 - Organizations, IAM Identity Center, Amplify, AppSync, Cognito, S3, Route 53
 - **Firebase**
 - Authentication, Firestore, Storage, Analytics, Crashlytics, App Distribution, Remote Config
- インフラ / IaC
 - Terraform, tfint, dotenvx, OIDC

フロントエンド / モバイル

- **Web アプリ開発**
 - フレームワーク: React, Next.js, Astro, TanStack Start
 - ルーティング / データ取得: TanStack Router, TanStack Query
 - UI: shadcn/ui, Tailwind CSS
- **モバイルアプリ開発**
 - クロスプラットフォーム: React Native (Expo), Flutter
 - ネイティブ: Swift (SwiftUI, UIKit)
- **アーキテクチャ**
 - モノレポ設計 (Bun workspaces), Clean Architecture, レイヤーアーキテクチャ, MVVM, Redux, Riverpod
 - 独自アーキテクチャ 「[Framework-Independent Architecture \(FIA\)](#)」の考案・公開 ([スライド](#) / [YouTube](#))

品質 / CI/CD

- テスト
 - ユニット / 結合: Vitest, Swift Testing, XCTest, Quick/Nimble
 - E2E: Playwright, Maestro
 - UI / モック / アクセシビリティ: Storybook, MSW, @axe-core/playwright
- **コード品質**
 - Lint / フォーマット: Biome, oxlint, oxfmt, ESLint, Prettier
 - 依存 / 構成の検査: knip, dependency-cruiser

- コミット / Git フック: commitlint, lefthook, husky
- 依存更新: Renovate
- **CI/CD**
 - CI: GitHub Actions, Xcode Cloud, Bitrise
 - リリース / デプロイ: release-please, Wrangler
- **SEO / パフォーマンス**
 - Core Web Vitals, Lighthouse, PageSpeed Insights, Google Search Console
 - 構造化データ (JSON-LD / Schema.org), OGP 自動生成 (satori / sharp), sitemap / RSS / canonical
 - AI クローラー対応 (llms.txt, Content-Signal 対応 robots.txt), Pagefind (静的サイト内全文検索)

プロセス / OSS

- **プロジェクト管理**
 - 役割: Scrum Master 経験
 - アジャイル開発: Jira, Confluence, GitHub Projects, Trello, Zenhub, Notion, Backlog, Linear
- **OSS 開発 (自作 CLI ツール)**
 - [ai-chat-md-export](#) (AI チャット履歴の Markdown 変換)
 - [mermaid-markdown-wrap](#) (Mermaid の Markdown ラップ)
 - [issue-linker](#) (GitHub Issue 参照の検証)
 - [readme-i18n-sentinel](#) (README 翻訳の構造検証)
 - 配布: npm / Homebrew / GitHub Actions Marketplace / 実行バイナリ。release-please・Codecov などの品質管理を全ツールで統一

職務経歴

株式会社ZENSHIN (2026年04月 - 現在)

- 2026年
 - [No.11] [株式会社ZENSHIN](#) - CTO (IT コンサルティング / Sler / SES)
 - 顧客向け AI 活用支援・AI ツール開発 (FDE) / 社内 AI システム開発 / コーポレートサイト・社内インフラ

フリーランス (2021年07月 - 2026年03月)

- 2025年
 - [No.10] ショートドラマアプリ開発 - フロントエンドエンジニア (React Native / Next.js) (エンタメ業界 G社)
 - [No.9] NFT ゲームアプリ開発 - Flutter リードエンジニア (WEB3 特化 Sler F社)
- 2024年
 - [No.8] SNS アプリ開発 - iOS リードエンジニア (Sler E社)
- 2023年
 - [No.7] マーケティングリサーチアプリ開発 - iOS リードエンジニア (スタートアップ D社)
- 2022年
 - [No.6] ファンクラブアプリ開発 - iOS エンジニア (メガベンチャー C社)
- 2021年
 - [No.5] ドローン制御アプリ開発 - iOS エンジニア (ドローンベンチャー B社)

外資系ITコンサル A社 (2017年04月 - 2021年06月)

- 2021年
 - [No.4] ショッピングアプリ開発 - Flutter エンジニア
 - [No.3] 飲食店管理アプリ開発 - iOS リードエンジニア
 - 2018年 - 2020年
 - [No.2] クレジットカードアプリ iOS アプリ開発 - iOS エンジニア
 - [No.1] クレジットカードアプリ API 開発 - サーバーサイドエンジニア
-

[No.11] 株式会社ZENSHIN - CTO (IT コンサルティング / Sler / SES)

チーム体制

- CTO として、顧客向けの AI 活用支援から社内プロダクト・インフラの開発・運用までを一人で担当

案件概要・担当業務

- 創業フェーズの株式会社ZENSHIN の CTO として、以下 6 つの業務を並行して推進
 - **[業務1] AI 活用支援 / 提案活動 (FDE)** — AI 活用の相談・提案から実装・導入まで支援
 - **[業務2] AI マッチングシステム開発** — Cloudflare フルスタックの RAG / MCP / AI エージェント基盤
 - **[業務3] コーポレートサイト開発** — <https://www.zenshin-inc.co.jp/> の設計・構築・運用
 - **[業務4] 社内インフラ管理** — Terraform によるマルチクラウド IaC
 - **[業務5] AI 案件選別システム開発** — 外部案件を LLM で自動評価する社内システム
 - **[業務6] 社外向け資料共有基盤の開発** — Cloudflare Access 認証付きの資料配信基盤

経験した技術

- **FDE / AI 活用支援 (業務1)**
 - v0 や Claude Code による商談前の高速プロトタイピング
 - LLM ツールの使い分け指導、プロンプト改善、MCP 化・Skills 設計の方針策定
 - 建設業向けに施工計画書の AI 作成支援システムを開発し、実案件データで運用
- **RAG / ベクトル検索マッチング (業務2)**
 - Workers AI (bge-m3) + Vectorize による意味ベースの類似検索
 - ベクトル検索 (一次絞り込み) → AI エージェント採点 (二次精査) の多層パイプライン設計
 - 自作の評価スクリプトで LLM モデルを比較検証し、シーンごとに使い分け
- **MCP サーバー / AI エージェント運用 (業務2)**
 - 70 以上のツールを持つ MCP サーバーの開発 (MCP TypeScript SDK v2, Better Auth OAuth 認証・マルチテナント認可)
 - WebMCP (ブラウザ内 MCP) の Origin Trial 先行導入
 - 50 以上の Claude Code スキルと定期ルーチンによる業務自動化 (Claude Code / Codex 両対応)
- **LINE 公式アカウント基盤 (業務2)**
 - 友だち紐付け不要の本人アカウント連携と、スタッフ向けの双方向チャット
 - 高スコア案件の本人 LINE への自動提案、Quick Reply による稼働状況の定期ヒアリング
 - MCP 経由で AI エージェントが候補者への返信・提案配信まで実行 (人間の承認・監査記録付き)
- **Cloudflare フルスタック (業務2・3・5・6)**
 - Workers / Workflows / Queues / D1 / Vectorize / Browser Rendering によるサーバーレス構成
 - Hono + oRPC + React 19 / TanStack Start による Web アプリ、Astro によるコーポレートサイト (Lighthouse 100 を維持)
 - Email Sending + Slack Interactive による採用応募対応の自動化、Access 認証付きの資料配信
- **マルチクラウド IaC (業務4)**
 - GCP / Cloudflare / AWS / Google Workspace を Terraform で横断管理
 - Workload Identity Federation (OIDC) による鍵レス認証、差分ベースの terraform plan CI

取り組み・貢献

- **営業が AI エージェントで実業務を回す状態の実現**
 - OAuth 認証付きの MCP を整備し、IT 知識のない営業が自然言語で要員検索～人材紹介まで完結
 - 案件 1,000 件 × エンジニア 1,000 名の規模から、5 分以内に適切なマッチングを提示
- **AI の判断が顧客接点まで届く自動ループの構築**
 - 案件収集 → AI 採点 → 本人 LINE への自動提案 → 本人回答まで、人手を介さない運用を実現
 - 送信時間帯・連投抑制・人間の承認と監査記録など、安全に運用するためのガードを設計
- **AI を過信しない設計・運用**
 - 条件ミスマッチはサーバー側で決定論的にスコア上限を強制し、AI の過大評価を防止
 - 本番の「AI 判定 vs 人間判断」の乖離分析でプロンプトを継続改善 (25 回超の改修サイクル)
 - 構造化に AI を使わない決定論パースの採用など、AI と決定論処理の使い分けを徹底
- **一人でも回る運用の仕組み化**

- インフラを Cloudflare 中心で完結させ、小規模でも維持できる運用コストを実現
- 社内インフラを徹底的に IaC 化し、コード変更 → PR → plan CI → apply のフローを確立
- 採用しなかった選択肢を含め、技術的な意思決定の根拠をリポジトリに記録する文化を形成

開発環境

フロントエンド

- Astro, React 19, TanStack Start / Router / Query, shadcn/ui, TypeScript, Tailwind CSS v4, Bun

バックエンド / AI システム

- Hono, oRPC, Drizzle ORM, Zod, Better Auth, @modelcontextprotocol/sdk v2, WebMCP
- Cloudflare Workers AI, AI Gateway, Vectorize, Workflows, Queues, Browser Rendering, Email Sending
- LINE Messaging API, Slack API, Google Drive API, GitHub App 連携
- Python (uv), pandoc + LibreOffice (FDE 案件ツール開発)

クラウド / インフラ

- Cloudflare, GCP, AWS, Terraform, tfint, dotenvx, mise

CI/CD / テスト / 品質

- GitHub Actions, release-please, Wrangler, Renovate
- Vitest, Playwright, Storybook, MSW, @axe-core/playwright, oxlint, oxfmt, knip, lefthook

AI ツール (提案活動・開発)

- Claude Code, Codex, v0, Chrome DevTools MCP

分析 / 開発ツール

- Google Search Console, PageSpeed Insights, PostHog, Cloudflare Analytics, VSCode

[No.10] ショートドラマアプリ開発 - フロントエンドエンジニア (React Native / Next.js)

チーム体制

- 案件全体人数 : 6名
 - フロントエンド (iOS + Android + Web) : 2名 (担当)

案件概要・担当業務

- 0 → 1 直後のグロースフェーズにおけるショートドラマアプリの開発
- React Native (Expo) による iOS / Android 同時開発を担当し、並行して Next.js による Web 版の開発も実施

経験した技術

- **React Native (Expo) / Next.js**
 - Monorepo 構成による iOS / Android / Web のクロスプラットフォーム開発
 - Expo SDK のメジャーバージョンアップ対応 (v52→v53、v53→v54、v54→v55)
 - Solito によるモバイル (Expo) / Web (Next.js) 間のコード共有
 - Tamagui を用いたプラットフォーム間の UI 統一
 - expo-iap によるアプリ内課金の実装
 - Rive を用いたアニメーションの組み込み
 - Next.js による Web アプリケーション開発
 - TanStack Query によるデータフェッチング・キャッシング
 - Crisp SDK を用いたカスタマーサポート用チャット機能の実装
- **開発プロセス改善**
 - Storybook による UI コンポーネントカタログの作成
 - Maestro によるモバイルアプリでの E2E テストの導入
 - Mise の導入による開発環境のセットアップの簡略化とバージョンの統一
- **AI・MCP活用**
 - Figma MCP を活用した Figma デザインからの実装
 - Storybook でコンポーネントを作成し、Google DevTools MCP / Playwright MCP で Claude Code による検証

- GitHub Actions による Claude Code 自動レビューの仕組みの構築

取り組み・貢献

- **ビジネス指標に基づくグロース施策の企画・実装**
 - ARPU・CTR・LTV などの KPI を CEO と連携しながら追い、ユーザー獲得コスト vs LTV を意識した施策設計を行った
 - ファネル分析で離脱ポイントを特定し、課金導線の改善によりサブスクの契約者数や課金アイテムの購入数を増加させた
 - ガチャ・ミッション機能・無料開放施策などの設計・実装を通じて、主要 KPI の改善に貢献した
 - オファアウォール連携を導入し、収益チャネルを拡大した
- **高速な PDCA サイクルの実践**
 - 施策立案から1週間でリリースする高速な PDCA サイクルをスプリントごとに繰り返し回した
 - A/B テストによる効果検証を行い、季節要因や曜日パターンなど複数の変数がある中で施策単体の効果を正しく測定した
 - 効果が見込めない施策は即座に機能削除し、効果がないという事実も意思決定に活かした
- **デザイナーとの協業**
 - 表示内容や画面遷移の導線、コンポーネントの色や配置といった UI/UX の仕様にまで踏み込んで改善案を提案した
 - 仮実装を素早く共有し、デザイナーと議論しながらリリース前に何度もブラッシュアップを重ねた
 - リリースごとの KPI の結果をデザイナーと共有し、データに基づいた改善提案を繰り返すことで提案の精度を高めていった

開発環境

React Native (Expo) / Next.js

- **アーキテクチャ:**
 - Monorepo 構成 (Expo + Next.js)
- **主要ライブラリ:**
 - TanStack Query, ts-proto, Tamagui, Solito, Expo Modules API (Swift / Kotlin)
- **コード品質・テスト:**
 - Storybook, ESLint, Prettier, Vitest, Maestro

Firebase / Google Cloud

- Firebase (Auth, Distribution, Remote Config, Analytics)
- Google Cloud (Cloud Run)

分析ツール

- BytePlus DI, Looker Studio, Adjust

CI/CD

- GitHub Actions

プロジェクト管理

- GitHub Projects, Notion, Linear

開発ツール

- VSCode, Android Studio, Xcode, GitHub Copilot, Claude Code, Codex, Gemini

デザインツール

- Figma

[No.9] NFT ゲームアプリ開発 - Flutter リードエンジニア (Flutter)

チーム体制

- 体制
 - PdM : 1名
 - PM : 1名
 - デザイナー : 1名
 - サーバーサイドエンジニア : 2名

- Flutter エンジニア : 1名 (担当)

案件概要・担当業務

- 0 -> 1 フェーズでの NFT ゲームアプリの開発における Flutter エンジニアを担当
- 唯一の Flutter エンジニアとして、アーキテクチャ考案・ライブラリ選定からすべての実装までを担当
- PM・デザイナー・サーバーサイドチームとの仕様調整も担当

経験した技術

- **Flutter**
 - Riverpod と Hooks を用いた状態管理
 - Google Maps API の活用と google_maps_flutter , geolocator を用いた地図表示と位置情報の取得
 - go_router を用いた画面遷移の実装
 - openapi_generator を用いた API クライアントコードの自動生成
 - dio を用いた HTTP 通信およびインターセプターによる JWT 認証の実装
 - flutter_secure_storage を用いたセキュアなデータ保存
 - permission_handler を用いた位置情報取得、写真撮影、写真フォルダへのアクセスの実装
 - slang による多言語対応
 - pedantic_mono によるコード品質の向上
 - ThemeData よりデザインシステムの実装
 - fvm による Flutter バージョンの管理
- **開発プロセス改善**
 - Prism を活用した API モックサーバーの構築
 - Lefthook による pre-commit 時の静的解析実行

取り組み・貢献

- **開発体制の改善活動**
 - GitHub Projects を活用したスクラムボードを作成し、タスクの進捗状況を可視化した
 - デイリーを開催し、毎日、メンバー間での情報共有と開発プロセスの改善を行った
 - バグの発見から修正までのプロセスを整備して、それをチーム内で運用した
- **API インターフェース設計と UI 先行開発**
 - サーバーサイドの Pull Request をレビューし、開発段階で API インターフェースの改善点をフィードバックを行った
 - OpenAPI 形式の yaml ファイルから Prism でのモックサーバーでの開発環境を整備して、UI の先行開発を実施した
- **デザインシステムの導入**
 - デザイナーと協力してデザインシステムを設計し、デザインの一貫性を実現した
 - デザインシステムを Flutter の ThemeData を通じて定義し、UI の実装コードを削減した

開発環境

Flutter

- **アーキテクチャ:**
 - Riverpod + Hooks による状態管理
- **主要ライブラリ:**
 - go_router, dio, slang, permission_handler, flutter_secure_storage, pedantic_mono, freezed, google_maps_flutter, geolocator, openapi_generator, fvm

CI/CD

- GitHub Actions

プロジェクト管理

- GitHub Projects, Notion, Slack

開発ツール

- VSCode, Android Studio, Xcode, GitHub Copilot, ChatGPT

デザインツール

- Figma
-

[No.8] SNS アプリ開発 - iOS リードエンジニア (Swift)

チーム体制

- 案件全体人数 : 約10名
 - iOS エンジニア : 1名 (担当)

案件概要・担当業務

- 0 → 1 フェーズでの SNS アプリ開発の立ち上げ案件
- 唯一の iOS エンジニアとして、アーキテクチャ考案・ライブラリ選定・CI/CD 構築からすべての実装までを担当
- PM・デザイナー・サーバーサイドチームとの仕様調整も担当

経験した技術

- **Swift**
 - Xcode 16 Beta での Strict Concurrency を含む Swift 6 対応
 - AVFoundation を活用した録音/再生の機能実装
 - [WhisperKit](#), [Speech](#) SDK を活用した音声データの文字起こしの実装
- **開発プロセス改善**
 - [Swift OpenAPI Generator](#) による API 通信処理の自動生成の GitHub Actions パイプラインの構築
 - [Swagger UI Action](#) を用いた API 仕様書の自動生成の GitHub Actions パイプラインの構築
 - [tbls](#) を用いた MySQL のテーブル定義書の自動生成の GitHub Actions パイプラインの構築
 - [pixelmatch](#) による View のスナップショットの差分検出の実装

取り組み・貢献

- **アジャイル開発**
 - テスタブルなアーキテクチャの導入:
 - モックで API レスポンスを差し替えられるアーキテクチャを導入し、API 提供前から実装を可能にした
 - デバッグ画面の作成:
 - iOS アプリに検証用のデバッグ画面を作成し、新機能や View の早期検証を可能にした
 - Docs as Code の導入:
 - [Swagger UI Action](#) や [tbls](#) によるドキュメント生成方法を調査して、サーバーサイドチームに展開した
- **CI/CD 環境の構築**
 - Xcode Cloud 導入:
 - Xcode Cloud で PR マージをトリガーに TestFlight 配信を自動化し、新機能の迅速な検証を可能にした
 - API インターフェース変更の自動 Pull Request 作成:
 - OpenAPI の変更をトリガーに、iOS リポジトリへ自動 Pull Request を作成する GitHub Actions を構築した
 - スナップショット差分テスト:
 - View のスナップショット差分テスト環境を構築し、不具合の早期発見を実現した
- **iOS メンバーの増員や引き継ぎを見越した GitHub 管理**
 - ドキュメント整備:
 - 環境構築手順、ライブラリ選定理由、アーキテクチャ、CI/CD 構成図、ブランチ戦略などを README に記載した
 - プロジェクト管理:
 - リリースノート、タグ、マイルストーン、GitHub Projects を整備し、タスクの進捗を時系列で振り返れるように管理した
- **実装・最新技術**
 - 実装:
 - ワイヤフレーム段階でのデザインを基に iOS アプリを実装し、実装の課題や仕様の課題を早期発見し、チームへ共有した
 - コード生成:

- View 層や UseCase 層のテストコードを含めたボイラープレートコードは [Sourcery](#) や [Mockolo](#) によって自動生成し、開発効率を高めた
- Swift 5 → Swift 6 への移行:
 - 早い段階から Beta 版 Xcode を用いて Swift 6 への移行を検証し、大きなトラブルなくスムーズに移行を完了した

開発環境

Swift

- アーキテクチャ:
 - Clean Architecture x Swift Package Manager でのマルチモジュール構成
- Swift 標準 SDK & API:
 - SwiftUI, Swift Package Manager, Swift Concurrency, Combine, AVFoundation, Speech, Swift Testing, String Catalogs, Swift OpenAPI Generator

CI/CD

- Xcode Cloud, GitHub Actions, Renovate

プロジェクト管理

- GitHub Projects, Notion, Backlog

デザインツール

- Figma

[No.7] マーケティングリサーチアプリ開発 - iOS リードエンジニア (Swift)

チーム体制

- 案件全体人数 : 約15名
 - iOS エンジニア : 3名 (iOS リードエンジニア担当)

案件概要・担当業務

- スタートアップ企業の 0 → 1 フェーズでのマーケティングリサーチサービスの立ち上げ案件
- toC 向けコンテンツ配信アプリと toB 向け BI ツールの 2 サービス構成で、iOS チームのリードエンジニアを担当

経験した技術

- Swift
 - iOS16 以上を対象 OS とした SwiftUI での画面開発
 - Clean Architecture x Swift Package Manager でのマルチモジュール構成の構築
 - Xcode Cloud での CI/CD 環境の構築
 - Protocol Buffers に対応した [SwiftProtobuf](#) のライブラリを用いたデータ連携
 - async/await, AsyncStream, TaskGroup, Actor などを用いた Swift Concurrency による非同期処理のハンドリング
 - [AWS Amplify SDK](#) を用いた Cognito での SMS での認証・認可、AppSync による GraphQL 疎通、Pinpoint によるログイベント送信、S3 とのデータ連携
 - デザインシステムを活用した画面実装
 - AVFoundation を用いた動画の再生
 - ReplayKit を用いた画面のレコーディング
 - 視線や感情の時系列データの Combine を用いたハンドリング
 - JavaScript を用いたアプリ内 WebView のイベントハンドリング
- 開発プロセス改善
 - リリース tag・リリースノート・レビュー追加などを自動化する GitHub Actions Workflow の実装
 - [Renovate](#) によるライブラリの自動更新 PR の作成の環境構築
 - [Periphery](#) による Swift コードの不要なコードの静的解析
 - Swift-DocC による iOS アプリのドメイン層のドキュメント化
 - [Mockolo](#) によるテスト用の Mock の自動生成
 - GitHub Copilot, ChatGPT の活用

取り組み・貢献

- 0 → 1 フェーズの iOS リードとして、アーキテクチャ・ライブラリ選定、ブランチ戦略、リリース手順の確立を担当した
- CI/CD 環境の構築や、iOS チームのスクラムボード運用の設計も行った
- チームに経験者のいない AWS Amplify SDK や SwiftProtobuf は、サンプルアプリで先行検証してチームに展開し、採用につなげた
- 視線分析・感情分析の SDK を、入れ替えても影響範囲が最小になるアーキテクチャで組み込んだ
- PdM・デザイナー・サーバーサイド・データ分析チームと、仕様やデータ連携インターフェースを調整した
- iOS チームの issue 運用を担当し、メンバーのタスクが途切れないよう先回りして行動し続けた

開発環境

Swift

- **アーキテクチャ:**
 - VIPER ベースの Clean Architecture x Swift Package Manager でのマルチモジュール構成
- **Swift 標準 SDK & API:**
 - SwiftUI, Swift Package Manager, Swift Concurrency, Combine, Swift-DocC, AVFoundation, Core ML, WebKit, ReplayKit, Logger
- **サードパーティ製 SDK:**
 - SwiftProtobuf, Firebase, Amplify, Nimble/Quick, LicensesPlugin, PhoneNumberKit, DeviceKit, SwiftFormat, SwiftGen, Lottie, Mockolo, Mint, Periphery

クラウド連携

- **AWS Amplify:**
 - AppSync (GraphQL), Cognito, S3, Pinpoint
- **Firebase:**
 - Crashlytics

CI/CD

- Xcode Cloud, GitHub Actions, Renovate

プロジェクト管理

- GitHub Projects, Notion, Backlog

インターフェース共有

- Protocol Buffers, Swagger

デザインツール

- Figma

[No.6] ファンクラブアプリ開発 - iOS エンジニア (Swift)

チーム体制

- 案件全体人数 : 約30名
 - iOS エンジニア : 5名 (担当)

案件概要・担当業務

- アーティストのファンクラブアプリにおけるスタンプラリー機能および景品交換の機能の開発を行なった
- デザイナーとの仕様の調整、見積もり、実装、レビュー、バグ修正を行なった

経験した技術

- Redux ベースのアーキテクチャ (VueFlux) での状態管理
- デザインシステムに沿った UI の実装

取り組み・貢献

- デザイナー、Android、Web フロントのエンジニアとコミュニケーションを取りながら、プラットフォーム間で仕様に大きな差がでない

ように開発した

開発環境

Swift

- **アーキテクチャ:**
 - Redux ベースのアーキテクチャ
- **Swift 標準 SDK & API:**
 - UIKit, AVFoundation
- **サードパーティ製 SDK:**
 - Carbon, VueFlux, ReactiveSwift, XcodeGen, Quick/Nimble, APIKit, CocoaPods, Carthage, Lottie

クラウド連携

- **Firebase:**
 - Crashlytics

CI/CD

- CircleCI, Fastlane

プロジェクト管理

- Wrike, Kibela

インターフェース共有

- Protocol Buffers, Swagger

デザインツール

- Figma

[No.5] ドローン制御アプリ開発 - iOS エンジニア (Swift)

チーム体制

- 案件全体人数 : 約15名
 - iOS エンジニア : 6名 (担当)

案件概要・担当業務

- BtoB 向けドローン制御アプリで、飛行の安定性改善・複数社ドローン対応・画面開発などを担当した
- アーキテクチャの検討、見積もり、実装、レビュー、バグ修正を行なった

経験した技術

- **Swift**
 - アーキテクチャの検討
 - Clean Architecture での実装
 - SwiftUI・UIKit x Combine を用いた画面実装
 - Swift Concurrency を用いた非同期処理の実装
 - Firebase Crashlytics、Xcode Organizer を用いたバグの原因調査
 - Logger API を用いたログ出力
 - Quick/Nimble ライブラリを用いた可読性の高いテストコードの記述
 - Mock を活用したテストコードの記述
 - iPad サイズ対応のアプリの実装
- **IoT**
 - 外部ライブラリを用いたドローンの制御の Swift での実装
 - PID 制御などの制御工学の理解と適切な制御モデルの Swift での実装
 - RoS(Robot Operating System) 環境の活用

取り組み・貢献

- Clean Architecture の採用により、UI を変えずにライブラリを差し替え、レイヤーごとの独立テストを実現した
- UIKit や Delegate パターンでの既存実装を、SwiftUI、Combine、Swift Concurrency といった新しい技術でリファクタリングした
- 以下のようなチームの運用の改善に積極的に取り組んだ
 - 見積会の実施
 - レトロスペクティブの実施
 - 開発チーム朝ハドル会の実施
 - プロダクトバックログを開発者が着手可能であることを表す「Ready」の概念の導入
 - Pull Request 提出から Merge までの運用ルールの見直し
 - リリースブランチ運用の見直し
 - デイリー前の Slack リマインダーの設定
 - デイリーでの相談事項の事前エントリー制の導入
 - Firebase Crashlytics 運用の見直し

開発環境

Swift

- **アーキテクチャ:**
 - VIPER ベースの Clean Architecture
- **Swift 標準 SDK & API:**
 - SwiftUI, UIKit, Combine, Swift Concurrency, Logger, MetricKit
- **サードパーティ製 SDK:**
 - Realm, Quick/Nimble, APIKit, CocoaPods, Carthage

クラウド連携

- **Firebase:**
 - Crashlytics, Analytics

CI/CD

- Bitrise, Fastlane

プロジェクト管理

- Zenhub

デザインツール

- Figma

[No.4] ショッピングアプリ開発 - Flutter エンジニア (Flutter)

チーム体制

- 案件全体人数 : 2名
 - Flutter エンジニア : 1名 (担当)
 - デザイナー : 1名

案件概要・担当業務

- Flutter での iOS・Android クロスプラットフォーム開発を採用したショッピングアプリのデモアプリの開発を担当
- Flutter でのフロントエンド実装から Firebase を活用したバックエンド実装まで、すべて一人で行った

経験した技術

- **Flutter**
 - Provider による状態管理
- **Firebase**
 - Authentication による認証
 - Firestore によるデータの永続化、NoSQL DB 設計
 - Storage への画像データの永続化

- Crashlytics によるクラッシュ報告管理
- App Distribution による iOS・Android のアプリ配布
- Analytics による KPI 指標の集計
- Google Maps API での地図活用
- **CI/CD**
 - Codemagic での iOS・Android のアプリ配布の自動化
 - Fastlane から App Distribution への配布

取り組み・貢献

- Firebase を活用したサーバーレス構成でのモバイルバックエンドサービスの設計・実装を行った
- Flutter による Android 対応（マテリアルデザイン、Google Play ストアでの配信）を経験した

開発環境

Flutter

- Provider

クラウド連携

- Firebase:
 - Authentication, Firestore, Storage, Crashlytics, App Distribution, Analytics
- Google Maps API

CI/CD

- Codemagic, Fastlane

デザインツール

- Adobe XD

[No.3] 飲食店管理アプリ開発 - iOS リードエンジニア（Swift）

チーム体制

- 案件全体人数：約10名
 - iOS エンジニア：3名（リードエンジニア担当）

案件概要・担当業務

- BtoB 向け飲食店管理モバイルアプリの MVP アプリの作成
- iOS リードエンジニアとして、要件の調整やサーバーサイドチームとの API インターフェースの検討などを行った
- Scrum Master も兼任した

経験した技術

- **Swift**
 - SwiftUI での画面実装
 - Codable プロトコルを用いた JSON の変換
 - TestFlight によるアプリ配信
 - アーキテクチャ、ディレクトリ構成の検討
- **Scrum Master**
 - スクラムボードの設計
 - 会議のファシリテーション

取り組み・貢献

- 決められた要件をただ実装するだけでなく、お客様やデザイナーによりよい仕様やデザインを提案した
- スクラムボードの扱いやコードレビュー方法などを、レトロスペクティブを待たずチーム内で相談し常に改善した

開発環境

Swift

- **アーキテクチャ:**
 - MVVM
- **Swift 標準 SDK & API:**
 - SwiftUI
- **サードパーティ製 SDK:**
 - SwiftLint

プロジェクト管理

- Zenhub, Trello

デザインツール

- Adobe XD

[No.2] クレジットカードアプリ iOS アプリ開発 - iOS エンジニア (Swift)

チーム体制

- 案件全体人数 : 約20名
 - iOS エンジニア : 4-5名 (担当)

案件概要・担当業務

- BtoC 向けのクレジットカードアプリの iOS アプリの開発における見積もり、実装、テスト、レビュー、バグ修正を担当した
- メインはサーバーサイドチームの担当であったが、作業の手が空いたり、iOS チームの負荷が上がったときに iOS チームを担当した

経験した技術

- UIKit での画面実装
- API 疎通
- Realm でのデータ永続化
- XCTest でのテストコード実装
- MVVM での実装
- Delegate パターンの実装
- Human Interface Guidelines に基づいた UI 実装
- Moneytree LINK SDK といったサードパーティー製のライブラリの組み込み

開発環境

Swift

- **アーキテクチャ:**
 - MVVM
- **Swift 標準 SDK & API:**
 - UIKit
- **サードパーティ製 SDK:**
 - CocoaPods, Carthage, Realm, Moneytree LINK SDK

通信キャプチャ

- mitmproxy

プロジェクト管理

- Jira, Confluence, Trello

デザインツール

- Sketch, InVision

[No.1] クレジットカードアプリ API 開発 - サーバーサイドエンジニア (PHP)

チーム体制

- 案件全体人数：約20名
 - サーバーサイドエンジニア：4名（担当）

案件概要・担当業務

- BtoC 向けクレジットカード明細管理アプリのリニューアルに伴い、API やバッチの開発
- サブリードディベロッパーとして、お客様向け説明資料の作成、設計、見積もり、実装、テスト、レビューを担当
- チーム結成から約 2 年半、リリースしたシステムを継続的にアップデートした

経験した技術

- **API 設計/開発**
 - PHP での API 設計・開発
 - MySQL での DB 設計・開発
 - OAuth2.0 での認証・認可の実装
- **バッチ設計/開発**
 - Java でのバッチの設計・開発
- **テスト**
 - API の単体・結合テストの設計と実装
 - Postman での API テストの自動化
 - JMeter での負荷テスト
- **ドキュメンテーション**
 - OpenAPI (Swagger) でのインターフェース設計・共有
 - PlantUML での設計

取り組み・貢献

- モバイルアプリケーションのバックエンド開発における設計、実装、テスト、リリース、運用までのフルライフサイクルを経験できた
- iOS チームと兼任していたため、モバイルアプリからの視点を API のインターフェースの設計に取り込むことができた
- API の結合テストを Postman によって自動化することで、少ない工数で網羅的に繰り返しテストを実施し、品質を担保することができた

開発環境

使用言語

- PHP (CodeIgniter)
- Java

プロジェクト管理

- Jira, Confluence, Trello

テストツール

- Postman, JMeter

ドキュメンテーション

- OpenAPI, PlantUML, draw.io
-